AnalysisTools user manual

RNDr. Karel Šindelka, Ph.D.
k.sindelka@gmail.com

Version 4.0 (Released ~~August 18, 2020~~ at some point, it will be...)

Contents

1 Introdution **2**
1.1 Installation . . . 2

2 Format of input/output files **3**
2.1 Structure files . . . 3
2.1.1 VTF format . . . 3
2.1.2 FIELD format . . . 4
2.1.3 XYZ format . . . 5
2.1.4 LAMMPS data format . . . 5
2.1.5 LAMMPS lammpstrj format . . . 5
2.2 Coordinate files . . . 6
2.2.1 VTF format . . . 6
2.2.2 XYZ format . . . 6
2.2.3 LAMMPS lammpstrj format . . . 7
2.3 Aggregate file . . . 7

3 Utilities **9**
3.1 Info . . . 10
3.2 SelectedVcf . . . 11

4 Computational details **13**
4.1 Commonly used structures . . . 13
4.1.1 Box . . . 13
4.1.2 Counts . . . 13
4.1.3 BeadType . . . 14
4.1.4 Bead . . . 14
4.1.5 MoleculeType . . . 15
4.1.6 Molecule . . . 16
4.1.7 Aggregate . . . 16
4.2 Read system data . . . 16
4.2.1 VtfReadStruct . . . 16
4.2.2 VtfReadTimestep . . . 19

1. Introdution

AnalysisTools is a set of utilities to (mainly) analyse trajectories of particle-based simulations. The 34 utilities (including one from the DL_MESO simulation package can be roughly divided into four types:

- utilities to calculate system-wide properties; e.g., pair correlation functions or particle densities along a simulation box axis
- utilities to calculate per-molecule or per-aggregate properties (where aggregate stands for any supramolecular structure); e.g., shape descriptors for individual molecules or whole micelles
- utilities to manipulate a configuration; e.g., create initial configuration for a simulation from scratch or by adding molecules to an existing one
- helper utilities to analyse text files; e.g., calculate averages and standard deviations of a data series

The `Examples` directory contains examples showcasing capabilities of individual utilities as well as possible workflows chaining several utilities.

1.1 Installation

AnalysisTools requires `C` and `Fortran` compilers and `cmake`. The utilities should be compiled in a separate directory, typically `<root>/build` (where `<root>` is the AnalysisTools root directory). The following generates a `Makefile` within `<root>/build`:

```
mkdir <root>/build; cd <root>/build; cmake ../
```

To then compile the utilities, simply run `make` (to compile all utilities) or `make <utilitiy name>` (to compile a single utility) in `<root>/build`. The binaries are located in the `<root>/build/bin` directory.

2. Format of input/output files

This section describes several file types used by many of the AnalysisTools utilities. Output files for the utilities themselves are described in their respective sections in Chapter 3.

Input files are divided into two categories: structure and coordinate files. The structure files contain information about the system composition, that is, number of beads and molecules, bead properties (mass, charge, etc.), molecule properties (number and types of beads, connectivity, etc.). The coordinate files contain individual timesteps with beads' coordinates (and, possibly, velocities, forces, etc.). Some files can be used as both a structure and a coordinate file, but using a different structure file may be essential. For example, an `xyz` file (see below) may be used as both, but because it only provides bead names, using a separate structure file would provide additional information (bead and/or molecule properties).

For working with aggregates, AnalysisTools' `agg` file format (see below) describes their compositions in terms of molecule numbers from a structure file used to generate it. Any of the `Aggregates` (TODO: reference) utilities can create the `agg` file.

2.1 Structure files

As structure files, any of the following formats can be used; example files for a simple 'sandbox' system are provided in the `Examples/InputFiles` directory. Utilities load a specific structure using the `-i <file>` option; if the option is missing, it is usually assumed the structure file is the same as the coordinate file.

`Info` utility (TODO: reference) can convert these formats between each other via the `-o <file>` option.

2.1.1 VTF format

See here for complete description and `in.vtf` and `in.vsf` for example files (a `vtf` file contains a structure part followed by a coordinate part, but a `vsf` file contains only the structure part). In this format, every bead is defined on its own line and all bonds all listed.

Not all keywords listed on the web page for the 'Atom lines' are recognized; keyword `n[ame]` is mandatory and optional keywords are `m[ass]`, `charge|q`, `r[adius]`,

resid, and res[name]. For the <aid-specifier>, only a single bead index number or the default keyword are recognized.

AnalysisTools groups beads (and molecules) into bead (and molecule) types. Generally, only name defines types of beads, i.e., should two beads share the name, they will be of the same type; their mass, charge, and radius will each equal to that for the bead type's topmost atom line containing the corresponding keyword. If the keyword is missing from all atom lines, that characteristic is undefined. The --detailed command line option triggers detailed recognition, where beads that share the name but differ in mass, charge, and/or radius will be of different type.

Different molecule types, on the other hand, are always distinguished based on all their characteristics, i.e., name, connectivity, and order of beads (order of its vsf indices). However, molecule types can also be affect by the --detailed option because of the bead type definition

For the effect of the --detailed option, see the Examples/Info directory.

For bond lines, only single bonds are recognized (i.e., bond <index1>:<index2> format).

Periodic boundary condition can be included as pbc <x> <y> <z> for a cuboid box or pbc <a> <b> <c> <alpha> <beta> <gamma> for a general paralleloid box.

2.1.2 FIELD format

A format similar to the FIELD input file for DL_MESO_DPD simulation package; see in.FIELD for an example. This concise format defines species of beads and molecules, providing number of beads/molecules of given species as well as their properties.

The species section is the same as in the DL_MESO FIELD file, but the molecules section differs slightly. Besides the beads part, every molecule can have bonds, angles, and dihedrals, but even though up to three parameters for potentials are read, AnalysisTools does not recognize the potential keywords (harm, cos, etc.). These potential parameters are only used for generating new structure files via the Info utility (TODO: reference). The molecules section can also contain one extra part: impropers for improper angles which has the same format as the dihedrals section (this was added because the LAMMPS molecular dynamics simulator distinguishes between the two).

Note that the FIELD file does not have to contain Interactions section; this part is ignored.

To be recognized by AnalysisTools utilities, the file must either be called FIELD or have a .FIELD extension.

2.1.3 XYZ format

This is the simplest and probably best known format, see, e.g., here for the description and `in.xyz` for an example. An `xyz` file is essentially a file with timesteps (bead coordinates), and the only structure information are bead names.

The comment line at the beginning of each timestep can contain periodic boundary conditions as `<x> <y> <z>` (cuboid box) or `<a> <b> <c> <alpha> <beta> <gamma>` (general paralleloid box); it is similar to the `pbc` keyword in `vtf` file. To use this box definition, use `-pbc <n>` option where `<n>` is the position of the first number on the comment line (e.g., `1` if the comment line starts with the box size).

2.1.4 LAMMPS data format

This rather complicated format comes from the LAMMPS molecular dynamics simulator and lists the numbers of beads, bonds, angles, etc., as well as individual beads, bonds, angles, etc., and information about potentials for bonds, angles, etc. See here for description of the format and `in.data` for an example.

AnalysisTools reads the header information as well as `Masses`, `Atoms`, `Velocities`, `Bonds`, `Angles`, `Dihedrals`, `Impropers`. It also reads `Bond Coeffs`, `Angle Coeffs`, `Dihedral Coeffs`, and `Improper Coeffs` sections.

The `Atoms` section must have `atom_style full` format, that is every bead line must have the format `<id> <mol id> <bead type> <q> <x> <y> <z>`, where `<id>` is a bead index, `<mol id>` is a molecule index (-1 means the bead is in no molecule), `<bead type>` is the bead type index corresponding to a line in the `Masses` section, `<q>` is charge, and `<x> <y> <z>` are coordinates.

For the `Bond Coeffs` and `Angle Coeffs` sections, harmonic potential is assumed, and the first value of each bond/angle type is multiplied by 2 because LAMMPS uses harmonic spring strength of $k/2$ (as opposed to, e.g., DL_MESO which uses k).

For the `Dihedral Coeffs` and `Improper Coeffs` sections, AnalysisTools reads at most three numbers without assuming any specific potential.

All the `Coeffs` sections are used only for generating new structure files via the `Info` utility, TODO: reference (specifically, only to generate `FIELD` and LAMMPS `data` files because no other supported file contains this information).

To be recognized by AnalysisTools utilities, the file must have an extension `.data`.

2.1.5 LAMMPS lammpstrj format

This mainly coordinate file format comes from the LAMMPS dump style custom command. See `in.lammpstrj` for an example.

Of the possible LAMMPS attributes, AnalysisTools recognizes only `id` (bead index – mandatory), `element` (bead name), `x y z` (bead coordinates), `vx vy vz` (bead velocities), and `fx fy fz` (bead forces). Any other attribute is ignored. Note that if `element` is missing, all beads are considered of the same type called `bt`.

To be recognized by AnalysisTools utilities, the file must have an extension `.lammpstrj`.

2.2 Coordinate files

As coordinate files, any of the following formats can be used; example files for a simple 'sandbox' system are provided in the `Examples/InputFiles` directory. These files can be combined with any of the above-described structure files (via the `-i <file>` option), but some may be used as both the structure and coordinate file.

`SelectedVcf` utility (TODO: reference) can generate any of these file types, converting between the formats; all utilities that require coordinate file can use any of these formats.

2.2.1 VTF format

See here for the description of a coordinate part of the `vtf` format and `in.vtf` and `in.vcf` for example files (a `vtf` file contains the coordinate part after a structure part, and a `vcf` file contains only the coordinate part).

AnalysisTools recognizes both ordered and indexed timesteps as well as a `pbc` line defining box dimensions.

If an AnalysisTools utility is not provided with an extra structure file (`-i <file>` option), the same filename is assumed but with the `.vsf` extension instead of the `.vcf` extension.

2.2.2 XYZ format

This is the simplest and probably best known format, see, e.g., here for the description and `in.xyz` for an example.

The comment line at the beginning of each timestep can contain periodic boundary conditions as `<x> <y> <z>` (cuboid box) or `<a> <b> <c> <alpha> <beta> <gamma>` (general paralleloid box); it is similar to the `pbc` keyword in `vtf` file. To use this box definition, use `-pbc <n>` option where `<n>` is the position of the first number on the comment line (e.g., `1` if the comment line starts with the box size).

The `xyz` coordinate file will be used as a structure file as well unless an extra structure file is provided via the `-i <file>` option.

2.2.3 LAMMPS lammpstrj format

This coordinate file format comes from the LAMMPS dump style custom command. See `in.lammpstrj` for an example.

Of the possible LAMMPS attributes, AnalysisTools recognizes only `id` (bead index – mandatory), `element` (bead name), `x y z` (bead coordinates), `vx vy vz` (bead velocities), and `fx fy fz` (bead forces). Any other attribute is ignored. Note that if `element` is missing, all beads are considered of the same type called `bt`.

To be recognized by AnalysisTools utilities, the file must have an extension `.lammpstrj`.

This coordinate file will be used as a structure file as well unless an extra structure file is provided via the `-i <file>` option.

2.3 Aggregate file

TODO: rewrite

A `file.agg` is generated using `Aggregates` (or `Aggregates-NotSameBeads`) utility. The file contains information about the number of aggregates in each timestep and which molecules and monomeric (i.e., unbonded) beads belong to which aggregate. It serves as an additional input file for utilities that calculate aggregate properties; `agg` file is, therefore, linked to the `vcf` file that was used to generate it.

The `agg` file is a simple text file. The first line contains the command used to generate it – parts of this command may be necessary for subsequent analysis of aggregates. The second line is blank, and from the third line, the data for individual timesteps are shown. It follows these rules:

- each timestep starts with `Step: <int>` (only `Step` keyword is read by the utilities)
- the second line is the number of aggregates in the given timestep and is followed by a blank line
- there are two lines for each aggregate:
  1) number of molecules in the aggregate followed by their indices taken from the `vsf` file (`resid` indices)
  2) number of monomeric beads in the aggregate followed by their indices taken from the `vsf` file (`atom` indices)
- no blank or comment lines are allowed inside the aggregate block
- not all molecules present in the `vcf` file used to generate this file must be present in every timestep

Note that the term aggregate also refers to free chains (i.e., fully dissolved chains).

When the keyword `Last` is present instead of `Step`, it signalises the end of the `agg` file; no utility will read anything beyond this keyword.

Besides using this file for further analysis by other utilities, the indices can be used in vmd to visualize, e.g., only a specific aggregate by using `resid 1 2 3` in the `Selected Atoms` box in vmd.

An example of an `agg` file can be found in the `Examples/DistrAgg` directory.

3. Utilities

All utilities have command line options affecting their behaviour. Most of the utilities share several standard command line options.

The following options are for all utilities and are not described with the utilities themselves.

General options common to all utilities	
`--verbose`	print information about all bead and molecule types to the screen
`--silent`	run silently, i.e., without any output at all (overrides `--verbose` option)
`--help`	print short description of the utility and usage
`--version`	print version of the utilities and exit

The following options are used by most utilities, and because they affect their behaviour, they are also listed in the utilities' descriptions.

Options for structure and coordinate input files	
`-i <name>`	use specified structure file, see Section 2.1 for possible formats
`--detailed`	only `vtf` structure file: detailed recognition for bead (and, consequently, molecule) types based on the bead names as well as charges, masses, radii
`--variable`	only `vtf` coordinate file: assume indexed timesteps with variable numbers of beads (otherwise, the same number of beads in each timestep is required)
`-pbc <n>`	only `xyz` file: position of the first box dimension in the file's comment line at the beginning of each timestep
`-st <n>`	any coordinate file: start calculations at `n`-th timestep
`-e <n>`	any coordinate file: end calculations at `n`-th timestep
`-sk <n>`	any coordinate file: skip every `n` timesteps during calculations

For `vtf` structure files, the bead types are defines solely based on their name; the other properties (mass, charge, and radius) are taken from the first `a[tom]` line with the bead of that name that contains the appropriate keyword. For example,

lines

```
atom default name A m 1
atom 1 name B
atom 2 name B m 2
atom 3 name A m 2
atom 4 name A
```

would define two bead type called `A` and `B` with masses equal to 1 and 2, respectively. Using the `--detailed` option would split the `A` beads into three types: `A` and `A_1` with masses equal to 1 and 2, respectively, and `A_2` with undefined mass (it remains undefined because there is an ambiguity—should the last bead have mass 1 or 2?). On the other hand, the `B` beads would be assigned the same bead type with mass equal to 2 (there is no ambiguity because only one mass is specified for the `B` beads). See `Examples/Info` directory for a more complete example.

For `vtf` coordinate file with indexed timesteps, all timesteps are expected to have the same number of beads (though not necessarily the same beads). If some timesteps contain different number of beads, `--variable` option must be used. The option is implemented because without knowing in advance the number of beads in a timestep, reading that timestep is slower (significantly so for large systems).

Plain `xyz` format provides only bead names and coordinates, but the comment line at the beginning of every timestep can store extra information. Here, only box dimensions are read, but because there are extended `xyz` formats containing more information, the `-pbc` option is used to specify where the box dimensions are (as well as to tell the utility there are box dimensions at all). For example, `-pbc 3` must be specified for the following comment line

`something more 10 10 10 90 60 30`.

This would define a paralleloid box with $a = b = c = 10$ and angles $\alpha = 90°$, $\beta = 60°$, and $\gamma = 30°$. If less than 6 numbers were provided, only the first three would be used to define a cuboid box.

3.1 Info

This utility prints the system information read from an input structure file, possibly augmenting it with information from another structure file (`-i[!] <file>` option) and/or pruning the system to contain only beads from a provided coordinate file (`-c` option). `Info` can also write this information into a new structure file of the given format (`-o option`).

Allowed output file types are `vtf` structure file (with the extension `.vsf`), LAMMPS `data` file (with the extension `.data`), and DL_MESO FIELD file (called `FIELD` or with the extension `.FIELD`). Coordinates (and, possibly, velocities) from the coordinate file are also used for output LAMMPS `data` file.

Usage:
`Info <input> <options>`

Mandatory argument	
`<input>`	input structure file
Options for input files	
`-i[!] <file>`	extra input structure file; see `Examples/Info` directory for the effect of `-i` and `-i!` options (default: None)
`-c <file>`	input coordinate file (default: None)
`--detailed`	only `vtf` structure file: specify recognition of bead (and molecule types) according to more than the bead name
`-pbc <n>`	only `xyz` file: specify position of box dimensions on a comment line (default: no box dimensions)
`--variable`	only `vtf` coordinate file: specify indexed timesteps with variable number of beads (only affects the results if `-st` is used)
`-st <n>`	any coordinate file: use `n`-th timestep
Options for output file	
`-o <file>`	output coordinate file (default: None)
`-def <bead>`	only `vtf` structure file: use bead type `<bead>` for the 'atom default' line (default:bead type with the most unbonded beads)
`--mass`	only `data` structure file: define LAMMPS atom types by mass but print per-atom charges in the `Atoms` section (default: atom types are the same as `Info`-recognized bead types)

3.2 SelectedVcf

This utility creates a new coordinate file containing only beads of specified types.

Using `--reverse` switch, the bead types can also be specified in reverse, i.e., bead types to be excluded from the output file(s).

Which timesteps are saved can be controlled using `-st` option for the first timestep to save and `-e` option for the last timestep to save; using `-sk <int>`, `<int>` steps are ignored per one step saved. If the option to save only specified timesteps (`-n`) is used, `-sk` is ignored. A maximum of 100 steps can be explicitly specified using the `-n` option.

There is also an option to remove periodic boundary conditions (i.e., to join molecules). Conversely, the simualation box can be wrapped (i.e., the periodic boundary conditions applied). If both `--join` and `-w` options are used, the simuation box is first wrapped and then the molecules are joined.

Lastly, the `--last` saves to output file(s) only the last step from the input coordinate file; it is useful, when only the last timestep is required, but the total number of timesteps in the input file is unknown.

Usage:

```
SelectedVcf <input> <output> <bead type(s)> <options>
```

Mandatory arguments	
`<input>`	input coordinate file
`<output.vcf>`	output coordinate file with
`<bead type(s)>`	bead type names to save (can be omitted if `--reverse` is used)

Options	
`--reverse`	reverse bead types to save, i.e., exclude `<bead type(s)>` instead of including them; if no `<bead type(s)>` are specified, all bead types are saved
`--join`	join molecules by removing periodic boundary conditions
`--wrap`	wrap simulation box (i.e., apply periodic boundary conditions)
`-st <n>`	starting timestep for calculation (default: 1)
`-e <n>`	ending timestep for calculation (default: none)
`-sk <n>`	number of steps skip per one used (default: 0)
`-n <int(s)>`	save only specified timesteps
`--last`	save only the last step
`-x <name(s)>`	exclude molecules of specified name(s) – do not use if `<output.vcf>` is further analysed
`-xyz <name>`	save coordinates to `xyz` file – does not take into account `-x` option

4. Computational details

This chapter will contain some information about how things are coded in the utilities.

4.1 Commonly used structures

4.1.1 Simulation box size

`typedef struct Box {<members>} BOX;`

member	explanation
`(VECTOR)Length`	simulation box length*
`(VECTOR)TriLength`	box length for triclinic box†
`(double)alpha`	α in a triclinic cell (or 90° for orthogonal cell)
`(double)beta`	β in a triclinic cell (or 90° for orthogonal cell)
`(double)gamma`	γ in a triclinic cell (or 90° for orthogonal cell)
`(double [3][3])Transform`	transformation matrix to transform between 'real' and 'triclinic' coordinates‡
`(double [3][3])inverse`	inverse of the transformation matrix

*for triclinic cells, it corresponds to a, b, and c (see https://docs.lammps.org/Howto_triclinic.html)
†lx, ly, and lz in https://docs.lammps.org/Howto_triclinic.html ‡see https://www.ruppweb.org/Xray/tutorial/Coordinatesystemtransformation.htm

4.1.2 Basic system information

TODO: update when I know how things are with file types other than `vtf`
`typedef struct Counts {<members>} COUNTS;`

member	explanation
`(int)TypesOfBeads`	number of bead types
`(int)TypesOfMolecules`	number of molecule types
`(int)TypesOfBonds`†	number of bond types
`(int)TypesOfAngles`†	number of Angle types
`(int)TypesOfDiherals`†	number of Dihedral types
`(int)Beads`*	total number of beads in a file with coordinates
`(int)Bonded`*	total number of beads in all molecules

(int)Unbonded*	total number of monomeric beads
(int)BeadsInVsf*	total number of all beads in the system
(int)Molecules*	total number of molecules
(int)Aggregates	total number of aggregates

*for vtf , this number (usually) corresponds to a vcf file, i.e., when some beads defined in a vsf file are missing from the vcf file, these beads are not (usually) included in this number
†if the jparameters are unknown (such as for a vsf file), this has a value of -1 TODO: why not just say 0?

4.1.3 Information about bead types

typedef struct BeadType {<members>} BEADTYPE;

member	explanation
(char[17])Name	bead type name (at most 16 characters)
(int)Number	number of beads of the given type
(bool)Use¶	true/false value whether these beads should be used in a calculation
(bool)Write¶	true/false value whether these beads should be written to an output coordinate file
(double)Charge	electric charge of the bead type (1000 if undefined)
(double)Mass	mass of the bead type (0 if undefined)
(double)Radius	radius of the spherical bead (0 if undefined)

¶ TODO: will be removed

- array size: Counts.TypesOfBeads

4.1.4 Information about individual beads

typedef struct Bead {<members>} BEAD;

member	explanation
(int)Type	bead type index corresponding to a struct BeadType array index
(int)Molecule	molecule index corresponding to a struct Molecule array index or -1 for an unbonded bead
(int)nAggregates¶	number of aggregates the bead is in
(int *)Aggregate¶	1D array with aggregate indices
(int)Index	index corresponding to an input file (e.g, vsf file)
(struct Vector)Position†	Cartesian coordinates
(struct Vector)Velocity†	velocity of the bead
(bool) InTimestep	is the bead in the present timestep?

¶ TODO: will be removed
† struct Vector contains members (double)x, (double)y, and (double)z

- array size: Counts.Beads
- array elements 0 to Counts.Unbonded contain monomeric beads
- array elements Counts.Unbonded+1 to Counts.Beads contain bonded beads

- BEAD is usually accompanied by an (int *)Index array (with size of Counts.BeadsInVsf) connecting in-code bead indices with input file bead indices, i.e.,
  Bead[<in-code index>].Index = <in-file index> and
  Index[<in-file index>] = <in-code index>
  - TODO: explain Index array usefulness

4.1.5 Information about molecule types

typedef struct MoleculeType {<members>} MOLECULETYPE;

member	explanation
(char[9])Name	bead type name (at most 8 characters)
(int)Number	number of molecules of the given type
(int)nBeads	number of beads in these molecules
(int *)Bead	array with bead in-code indices (i.e., corresponding to struct Bead array indices)
(int)nBonds	number of bonds in these molecules
(int **)Bond*	2D array with two bead indices of the connected beads and a bond type
(int)nAngles	number of angles in these molecules
(int **)Angle†	2D array with three bead indices of the beads in the angle and an angle type
(int)nDihedrals	number of dihedral angles in these molecules
(int **)Dihedral§	2D array with three bead indices of the beads in the dihedral and a diheral type
(int)nBTypes	number of bead types in these molecules
(int *)BType	array with bead type indices (i.e., corresponding to a struct BeadType array index)
(bool)InVcf¶	true/false value whether these molecules are present in the vcf file
(bool)Use¶	true/false value whether these molecules should be used in a calculation
(bool)Write¶	true/false value whether these molecules should be written to an output coordinate file
(double)Charge‡	total electric charge of the molecule type
(double)Mass‡	total mass of the molecule type

*size of nBonds×3: Bond[i][0 and 1] hold in-molecule bead indices (i.e., indices corresponding to MoleculeType[].Bead array) for bond i and Bond[i][2] holds bond type (or -1 if bond type is undefined); bond types are defined in a struct Params array
†size of nAngles×4: Angle[i][0 to 2] hold in-molecule bead indices (i.e., indices corresponding to MoleculeType[].Bead array) and Angle[i][3] holds angle type (or -1 if angle type is undefined); angle types are defined in a struct Params array
§size of nDihedrals×5: Dihedral[i][0 to 3] hold in-molecule bead indices (i.e., indices corresponding to MoleculeType[].Bead array) and Dihedral[i][4] holds dihedral type (or -1 if dihedral type is undefined); dihedral types are defined in a struct Params array
¶ TODO: will be removed
‡undefined if any of the included bead types have undefined charge/mass

- array size: Counts.TypesOfMolecules

4.1.6 Information about individual molecules

`typedef struct Molecule {<members>} MOLECULE;`

member	explanation
`(int)Type`	index of molecule type corresponding to a `struct MoleculeType` array index
`(int *)Bead`	array with in-code bead indices (i.e., corresponding to `struct Bead` array indices); size: `MoleculeType[].nBeads`
`(int)Aggregate`¶	index of an aggregate the molecule is in corresponding to a `struct Aggregate` array index
`(int)Index`	index corresponding to an input file (e.g, `resid <id>` in `vsf` file)

¶ TODO: will be removed

- array size: `Counts.Molecules`

4.1.7 Information about individual aggregates

TODO: this will be changed (I guess) `typedef struct Aggregate {<members>} AGGREGATE;`

member	explanation
`(int)nMolecules`	number of molecules in an aggregate
`(int *)Molecule`	array with in-code molecule indices (i.e., corresponding to `struct Molecule` array indices
`(int)nBeads`	number of bonded beads in an aggregate
`(int *)Bead`	array with in-code indices of the bonded beads (i.e., corresponding to `struct Bead` array indices)
`(int)nMonomers`	number of monomeric beads in an aggregate
`(int *)Monomer`	array with in-code indices of the monomeric beads (i.e., corresponding to `struct Bead` array indices)
`(double)Mass`	total mass of an aggregate (undefined if any of the molecules have undefined mass)
`(bool)Use`¶	`true`/`false` value whether this aggregate should be used in a calculation

¶ TODO: will be removed

- array size: `Counts.Aggregates`

4.2 Read system data

4.2.1 Read `vtf` structure file

This function reads an input `vsf` file line by line, identifying all beads and molecules present there. There are two modes that some utilities can use: bead types are defined either by name only (the default behaviour), or by name, mass, charge, and radius. In the latter case, the bead types that should share a name are renamed by appending `_#` (where `#` is 1, 2,..., etc.).

TODO: requires splitting into more functions—I guess...

```
void VtfReadStruct(parameters)
```

parameter	in/out	explanation
`(char) struct_file[]`	in	name of the input `vsf` file
`(bool) detailed`	in	should the bead type determination be based on more than just name?
`(COUNTS) Counts`	out	basic system information (section 4.1.2)
`(BEADTYPE) BeadType[]`	out	information about bead types (section 4.1.3)
`(BEAD) Bead[]`	out	information about individual beads (section 4.1.4)
`(int) Index[]`	out	array connecting internal and `vtf` bead indices
`(MOLECULETYPE) MoleculeType[]`	out	information about molecule types (section 4.1.5)
`(MOLECULE) Molecule[]`	out	information about individual molecules (section 4.1.6)

1) read `struct_file` line by line
   - save all information from `atom` and `bond` lines
     - only the first `atom default` line is saved
   - count
     - beads (i.e., identify the highest index in `atom <id>` lines), saving into `Counts.BeadsInVsf`
     - molecules, saving into `Counts.Molecules`
     - unique bead and molecule names (and save those names)
     - `atom` and `bond` lines (TODO: necessary to mention here?)
   - stop reading when
     - end of file or `timestep` line encountered
     - unrecognised line or coordinate line encounted and exit program with error
     - end of file or `timestep` line encountered
     - unrecognised line or coordinate line encounted and exit program with error
2) save number of unbonded and bonded beads into `Counts.Unbonded` and `Counts.Bonded`
3) identify bead types
   - save the number of types to `Counts.TypesOfBeads`
   - fill a temporary BeadType array

   a) if `detailed` is `true`, identify the types based on names, mass, charge, and radius
      i) create a new bead type for every `atom` line that has a unique combination of name, mass, charge, and radius
      ii) merge some of the bead types, specifically:
         - if a keyword is missing in one `atom` line but present in another does not count as a different type; e.g. beads
           ```
           atom 0 n x m 1 q 1
           atom 1 n x m 1
           ```
           are of the same type (with charge +1)
         - however, there can be ambiguities, so e.g., beads
           ```
           atom 0 n x m 1 q 1
           atom 1 n x m 1
           atom 2 n x m 1 q 0
           ```
           remain as three distinct types (with charges 0, `undefined`, and +1)

- but only some `atom` lines are ambiguous; e.g., beads
  ```
  atom 0 n x m 1 q 1
  atom 1 n x m 1
  atom 2 n x m 1 q 0
  atom 3 n x q 0
  ```
  are still of three types with the same charges as above and with mass 1 (there is no ambiguity as `atom 3` is the same as `atom 2` except for the undefined mass)
- note that sometimes the charge, mass, and/or radius can remain undefined even though there is exactly one well defined value; e.g., in case of `atom` lines
  ```
  atom 0 n x m 1 q 1
  atom 1 n x m 1
  atom 2 n x m 1 q 0
  atom 3 n x q 0
  atom 4 n x m 1 q 0 r 1
  ```
  beads sharing their type with `atom 4` will have well defined radius of 1 (i.e., `atom 2`, `3`, and `4`), while `atom 0` and `1` will have radius undefined as they have different charge to `atom 4`; three bead types are recognized here:
  A. type with $m = 1$, $q = 1$, and radius `undefined` (`atom 0`)
  B. type with $m = 1$ and charge and radius `undefined` (`atom 1`)
  C. type with $m = 1$, $q = 0$, and $r = 1$ (`atom 2`, `3`, and `4`)

b) if `detailed` is `false`, identify the types by name only
- mass, charge, and radius for each bead type is taken from the first `atom` line of the given name where the respective parameter is defined

4) fill a temporary Bead array and a temporary associated index (i.e., `Index`) array
- put unbonded beads first and the bonded beads after them
- if `detailed` is `false`, only bead name is checked to determine its type, otherwise its name, mass, charge, and radius are all checked

5) if `detailed` is `true`, rename bead types with the same name
- when several bead types share a name, the name remains unchanged for the first one, but `_#` is added to all subsequent ones (`#` goes from 1 to N, where N is the number of bead types with the same name)
- if the bead name would become too long (i.e., over 16 characters), it is shortened before the `_#` is added

6) identify molecules and molecule types
- save the number of types to `Counts.TypesOfMolecules`
- fill temporary MoleculeType and Molecule arrays
- molecules must share all information to be of the same type
  - molecule name
  - numbers of beads and bonds
  - connectivity (i.e., the bonds must be identical)
  - order of bead types (i.e., the order of the beads' indices in `struct_file`)

7) copy data from temporary arrays to `Bead`, `BeadType`, `Index`, `Molecule`, and `MoleculeType`

4.2.2 Read single timestep from `vtf` coordinate file

This function reads a single timestep (its preamble and the coordinate block) from an input `vcf` file, identifying what beads are present via `Bead[].InTimestep` flag.

`void VtfReadTimestep(parameters)`

parameter	in/out	explanation
`(FILE) vcf`	in	pointer to open `vcf` file
`(char) vcf_file[]`	in	name of the input `vcf` file
`(BOX) Box`	out	box dimensions (section 4.1.1)
`(COUNTS) Counts`	out	basic system information (section 4.1.2)
`(BEADTYPE) BeadType[]`	out	information about bead types (section 4.1.3)
`(BEAD) Bead[]`	out	information about individual beads (section 4.1.4)
`(int) Index[]`	out	array connecting internal and `vtf` bead indices
`(MOLECULETYPE) MoleculeType[]`	in & out	information about molecule types (section 4.1.5)
`(MOLECULE) Molecule[]`	out	information about individual molecules (section 4.1.6)
`(int) step_count`	in & out	counter of read timesteps

1) read `struct_file` line by line
   - save all information from `atom` and `bond` lines
     - only the first `atom default` line is saved
   - count
     - beads (i.e., identify the highest index in `atom <id>` lines), saving into `Counts.BeadsInVsf`
     - molecules, saving into `Counts.Molecules`
     - unique bead and molecule names (and save those names)
     - `atom` and `bond` lines (TODO: necessary to mention here?)
   - stop reading when
     - end of file or `timestep` line encountered
     - unrecognised line or coordinate line encounted and exit program with error
     - end of file or `timestep` line encountered
     - unrecognised line or coordinate line encounted and exit program with error
2) save number of unbonded and bonded beads into `Counts.Unbonded` and `Counts.Bonded`
3) identify bead types
   - save the number of types to `Counts.TypesOfBeads`
   - fill a temporary BeadType array

   a) if `detailed` is `true`, identify the types based on names, mass, charge, and radius
      i) create a new bead type for every `atom` line that has a unique combination of name, mass, charge, and radius
      ii) merge some of the bead types, specifically:
         - if a keyword is missing in one `atom` line but present in another does not count as a different type; e.g. beads
           ```
           atom 0 n x m 1 q 1
           atom 1 n x m 1
           ```
           are of the same type (with charge +1)

- however, there can be ambiguities, so e.g., beads
  ```
  atom 0 n x m 1 q 1
  atom 1 n x m 1
  atom 2 n x m 1 q 0
  ```
  remain as three distinct types (with charges 0, `undefined`, and +1)
- but only some `atom` lines are ambiguous; e.g., beads
  ```
  atom 0 n x m 1 q 1
  atom 1 n x m 1
  atom 2 n x m 1 q 0
  atom 3 n x q 0
  ```
  are still of three types with the same charges as above and with mass 1 (there is no ambiguity as `atom 3` is the same as `atom 2` except for the undefined mass)
- note that sometimes the charge, mass, and/or radius can remain undefined even though there is exactly one well defined value; e.g., in case of `atom` lines
  ```
  atom 0 n x m 1 q 1
  atom 1 n x m 1
  atom 2 n x m 1 q 0
  atom 3 n x q 0
  atom 4 n x m 1 q 0 r 1
  ```
  beads sharing their type with `atom 4` will have well defined radius of 1 (i.e., `atom 2`, `3`, and `4`), while `atom 0` and `1` will have radius undefined as they have different charge to `atom 4`; three bead types are recognized here:
  A. type with $m = 1$, $q = 1$, and radius `undefined` (`atom 0`)
  B. type with $m = 1$ and charge and radius `undefined` (`atom 1`)
  C. type with $m = 1$, $q = 0$, and $r = 1$ (`atom 2`, `3`, and `4`)

b) if `detailed` is `false`, identify the types by name only
- mass, charge, and radius for each bead type is taken from the first `atom` line of the given name where the respective parameter is defined

4) fill a temporary Bead array and a temporary associated index (i.e., `Index`) array
- put unbonded beads first and the bonded beads after them
- if `detailed` is `false`, only bead name is checked to determine its type, otherwise its name, mass, charge, and radius are all checked

5) if `detailed` is `true`, rename bead types with the same name
- when several bead types share a name, the name remains unchanged for the first one, but `_#` is added to all subsequent ones (`#` goes from 1 to N, where N is the number of bead types with the same name)
- if the bead name would become too long (i.e., over 16 characters), it is shortened before the `_#` is added

6) identify molecules and molecule types
- save the number of types to `Counts.TypesOfMolecules`
- fill temporary MoleculeType and Molecule arrays
- molecules must share all information to be of the same type
  - molecule name
  - numbers of beads and bonds
  - connectivity (i.e., the bonds must be identical)
  - order of bead types (i.e., the order of the beads' indices in `struct_file`)

7) copy data from temporary arrays to `Bead`, `BeadType`, `Index`, `Molecule`, and `MoleculeType`